SIMATS ENGINEERING

PYTHON PROGRAMMING LAB

Complete Assignment Report: Questions 1–7

Assignment 2 (CSA08) | Python 3

Prepared for: Python Programming Lab Assessment

Student: Harish Raghav

CSA0823

Q1: Decisions with Many Boundaries

Q2: Loops that Count Attempts

Q3: Engineer a Vehicle Registration Number Validator

Q4: Building Long Text: + or join()?

Q5: Functions, Parameters and Scope

Q6: Recursion and Its Limits

Q7: Diagnose, Redesign and Validate

Appendix A: README.md

Appendix B: requirements.txt, Git commit plan and repository link

Q1 – Decisions with Many Boundaries

Problem – State Electricity Subsidy Eligibility. A household receives a subsidy at 100 units or less; between 101 and 200 units it receives a partial subsidy only if it holds a ration card; above 200 units it receives none unless a member is above 80 years of age. The developer wrote the rule with several separate `if` statements and some households received two subsidies.

Diagnosis and fix. Separate `if` statements are all tested independently, so a household with 80 units and a ration card satisfies both "units ≤ 100" and "units ≤ 200 and ration card" (see the last column of the table). An `if–elif–else` chain tests the conditions in order and executes exactly one branch, and the chained comparison `101 <= units <= 200` states the slab directly. Eight households including every boundary (100, 101, 200, 201) were predicted before running; every `if–elif–else` result matched its prediction.

Design decision. Each slab is one branch of the chain with a single small conditional inside it (ration card or age). I prefer this shallow structure to deeply nested ifs or one long and/or condition: each slab can be read and changed on its own, whereas long combined conditions repeat the unit limits, which is how overlapping rules and double subsidies appear.

Alternative concept and complexity. A `match–case` statement (Python 3.10+) and a table of (upper limit, decision) pairs searched in a loop give identical decisions (asserted in the program). `if–elif–else` and `match–case` make at most three comparisons: O(1) time and O(1) space. The table needs up to k comparisons for k slabs: O(k) time per decision and O(k) space, but adding a slab is a data change rather than a code change.

Prediction table (predictions written before running)

Units	Ration card	Oldest age	Predicted	if–elif–else result	Separate-if result
80	Yes	40	Full subsidy	Full subsidy (matches)	Full subsidy + Partial subsidy
100	No	35	Full subsidy	Full subsidy (matches)	Full subsidy
101	Yes	50	Partial subsidy	Partial subsidy (matches)	Partial subsidy
101	No	50	No subsidy (no ration card)	No subsidy (no ration card) (matches)	none
200	Yes	60	Partial subsidy	Partial subsidy (matches)	Partial subsidy
201	Yes	81	Subsidy (age 80+ exception)	Subsidy (age 80+ exception) (matches)	Subsidy (age 80+ exception)
201	Yes	79	No subsidy	No subsidy (matches)	none
350	No	90	Subsidy (age 80+ exception)	Subsidy (age 80+ exception) (matches)	Subsidy (age 80+ exception)

Code screenshot

Q1 - Code

```
"""Q1 - Decisions with many boundaries (State Electricity Subsidy Eligibility)

Rule
  units <= 100        -> subsidy
  101 <= units <= 200 -> partial subsidy ONLY if the household holds a ration card
  units  > 200        -> no subsidy, UNLESS a member is above 80 years of age
"""

# ---------- (A) The BUGGY design: separate if statements ----------
def subsidy_buggy(units, ration_card, oldest_age):
    result = []
    if units <= 100:
        result.append("Full subsidy")
    if units <= 200 and ration_card:          # BUG: also true when units <= 100
        result.append("Partial subsidy")
    if units > 200 and oldest_age > 80:
        result.append("Subsidy (age 80+ exception)")
    return result                             # can hold TWO decisions
```

Q1 - Code

```python


# ---------- (B) The CORRECT design: if-elif-else with chained comparison ----------
def subsidy(units, ration_card, oldest_age):
    if units <= 100:
        return "Full subsidy"
    elif 101 <= units <= 200:
        return "Partial subsidy" if ration_card else "No subsidy (no ration card)"
    else:                                   # units > 200
        return "Subsidy (age 80+ exception)" if oldest_age > 80 else "No subsidy"


# ---------- (C) Alternative concept 1: match-case (Python 3.10+) ----------
def subsidy_match(units, ration_card, oldest_age):
    match units:
        case u if u <= 100:
            return "Full subsidy"
        case u if u <= 200:
            return "Partial subsidy" if ration_card else "No subsidy (no ration card)"
        case _:
            return "Subsidy (age 80+ exception)" if oldest_age > 80 else "No subsidy"


# ---------- (D) Alternative concept 2: table lookup, searched in a loop ----------
SLABS = [(100, "Full subsidy"), (200, "NEEDS_CARD"), (float("inf"), "NEEDS_AGE")]

def subsidy_table(units, ration_card, oldest_age):
    for upper, decision in SLABS:           # k slabs -> at most k comparisons
        if units <= upper:
            break
    if decision == "NEEDS_CARD":
        return "Partial subsidy" if ration_card else "No subsidy (no ration card)"
    if decision == "NEEDS_AGE":
        return "Subsidy (age 80+ exception)" if oldest_age > 80 else "No subsidy"
    return decision


# ---------- Predictions made BEFORE running (8 households, every boundary) ----------
# (units, ration_card, oldest_age, predicted correct decision)
HOUSEHOLDS = [
    (80,  True,  40, "Full subsidy"),               # card + low units: buggy gives 2
    (100, False, 35, "Full subsidy"),               # boundary
    (101, True,  50, "Partial subsidy"),            # boundary
    (101, False, 50, "No subsidy (no ration card)"),
    (200, True,  60, "Partial subsidy"),            # boundary
    (201, True,  81, "Subsidy (age 80+ exception)"),  # boundary
    (201, True,  79, "No subsidy"),
    (350, False, 90, "Subsidy (age 80+ exception)"),
]

if __name__ == "__main__":
    print(f"{'units':>5} {'card':>5} {'age':>3}  {'predicted':28} {'if-elif-else':28} OK ",
          "buggy (separate ifs)")
    for u, c, a, pred in HOUSEHOLDS:
        good = subsidy(u, c, a)
        ok = "yes" if good == pred else "NO"
        print(f"{u:>5} {str(c):>5} {a:>3}  {pred:28} {good:28} {ok:3} {subsidy_buggy(u, c, a)}")

    # all four implementations must agree for every case
    for u, c, a, pred in HOUSEHOLDS:
        assert subsidy(u, c, a) == subsidy_match(u, c, a) == subsidy_table(u, c, a) == pred
    print("\nAll implementations agree with the predictions.")
    print("Households that received TWO decisions with separate ifs:",
          sum(len(subsidy_buggy(u, c, a)) > 1 for u, c, a, _ in HOUSEHOLDS))
```

Execution output screenshot

Q1 - Execution output

```
$ python q1_subsidy.py
units  card age  predicted                  if-elif-else               OK  buggy (separate ifs)
   80  True  40  Full subsidy               Full subsidy               yes ['Full subsidy', 'Partial subsidy']
  100 False  35  Full subsidy               Full subsidy               yes ['Full subsidy']
  101  True  50  Partial subsidy            Partial subsidy            yes ['Partial subsidy']
  101 False  50  No subsidy (no ration card)  No subsidy (no ration card)  yes []
  200  True  60  Partial subsidy            Partial subsidy            yes ['Partial subsidy']
  201  True  81  Subsidy (age 80+ exception)  Subsidy (age 80+ exception)  yes ['Subsidy (age 80+ exception)']
  201  True  79  No subsidy                  No subsidy                 yes []
  350 False  90  Subsidy (age 80+ exception)  Subsidy (age 80+ exception)  yes ['Subsidy (age 80+ exception)']

All implementations agree with the predictions.
Households that received TWO decisions with separate ifs: 1
```

Q2 – Loops that Count Attempts

Problem – OTP verification at a bank kiosk. The kiosk allows three attempts and then locks the account for 30 minutes; a blank entry must not count as an attempt. Entries are supplied as a list so that the results are repeatable.

while design. The attempt counter is increased only after the blank check. `continue` skips a blank entry without touching the counter, and `pass` is a placeholder branch for a future "resend OTP" option. **for design.** `for attempt in range(3)` uses an inner `while True` loop that swallows blank entries and `break` to leave it as soon as a real entry arrives; the `else` clause of the for loop runs only if no attempt succeeded, so it locks the account.

While or for? A while loop is more natural because the number of passes is not fixed: blank entries do not consume an attempt and the limit can change. If the counter is increased in the wrong place (before the blank check), blank entries use up attempts and a customer who then types the right OTP is wrongly locked out; the last lines of the output demonstrate this bug.

Changed condition. After three wrong attempts a correct security answer grants two extra attempts, once only (the limit becomes 5). A for loop over range(3) cannot be extended once it has started, so the while design adapts more easily.

Alternative concept and complexity. A recursive function that calls itself for the next attempt gives the same answers. Time is O(a) for a attempts in every version; the loops use O(1) stack space whereas the recursive version uses O(a) stack frames. With a = 3 or 5 either is acceptable, but the loop is simpler and cannot reach the recursion limit.

Test cases (OTP = 482913)

Case	Entries	while version	for version
correct first time	['482913']	VERIFIED on attempt 1	VERIFIED on attempt 1
blank, then correct	['', '482913']	VERIFIED on attempt 1	VERIFIED on attempt 1
two wrong, then correct	['111111', '222222', '482913']	VERIFIED on attempt 3	VERIFIED on attempt 3
three wrong	['1', '2', '3']	LOCKED for 30 minutes	LOCKED for 30 minutes
blanks between wrong ones	['1', '', '2', '', '3', '482913']	LOCKED for 30 minutes	LOCKED for 30 minutes

Code screenshot

Q2 - Code

```
"""Q2 - Loops that count attempts (OTP verification at a bank kiosk)

Rules: 3 attempts, then lock for 30 minutes. A blank entry is NOT an attempt.
Entries come from a list so the program is repeatable (no keyboard needed).
"""

# ---------- 1. while version ----------
def verify_while(otp, entries):
    entries = iter(entries)
    attempts = 0
    while attempts < 3:
        try:
            entry = next(entries).strip()
        except StopIteration:
            return "CANCELLED (customer walked away)"
        if entry == "":
            continue                    # CONTINUE: skip blank, counter NOT touched
        attempts += 1                   # counter updated AFTER the blank check
        if entry == otp:
            return f"VERIFIED on attempt {attempts}"
        elif entry.lower() == "resend":
            pass                        # PASS: placeholder for a future 'resend OTP' feature
    return "LOCKED for 30 minutes"      # reached only after 3 real wrong attempts


# ---------- 2. for version with range(3) and loop-else ----------
```

Q2 - Code

```python
def verify_for(otp, entries):
    entries = iter(entries)
    for attempt in range(1, 4):
        while True:                    # inner loop swallows blank entries
            try:
                entry = next(entries).strip()
            except StopIteration:
                return "CANCELLED (customer walked away)"
            if entry:                  # non-blank -> counts as this attempt
                break
        if entry == otp:
            return f"VERIFIED on attempt {attempt}"   # (could also be 'break')
    else:                              # ELSE of the for loop: runs only if NO break
        return "LOCKED for 30 minutes"


# ---------- Changed condition: 2 extra attempts after a correct security answer ----------
def verify_extended(otp, entries, security_answer_ok):
    entries = iter(entries)
    limit, attempts, extended = 3, 0, False
    while True:
        if attempts == limit:
            if not extended and security_answer_ok:
                limit += 2             # grant two extra attempts, once only
                extended = True
                continue
            return "LOCKED for 30 minutes"
        try:
            entry = next(entries).strip()
        except StopIteration:
            return "CANCELLED (customer walked away)"
        if entry == "":
            continue
        attempts += 1
        if entry == otp:
            return f"VERIFIED on attempt {attempts} (limit was {limit})"


# ---------- Alternative concept: recursion ----------
def verify_recursive(otp, entries, attempt=1, max_attempts=3):
    if attempt > max_attempts:
        return "LOCKED for 30 minutes"
    entries = list(entries)
    while entries and entries[0].strip() == "":
        entries.pop(0)                 # drop blanks
    if not entries:
        return "CANCELLED (customer walked away)"
    entry = entries.pop(0).strip()
    if entry == otp:
        return f"VERIFIED on attempt {attempt}"
    return verify_recursive(otp, entries, attempt + 1, max_attempts)


# ---------- The 'wrong place' bug ----------
def verify_bug(otp, entries):
    """Counter incremented BEFORE the blank check: blanks wrongly use up attempts."""
    entries = iter(entries)
    attempts = 0
    while attempts < 3:
        attempts += 1                  # BUG: wrong place
        entry = next(entries, "").strip()
        if entry == "":
            continue
        if entry == otp:
            return "VERIFIED"
    return "LOCKED (wrongly - blanks were counted)"
```

Q2 - Code

```
93
94
95  if __name__ == "__main__":
96      OTP = "482913"
97      cases = {
98          "correct first time":        ["482913"],
99          "blank, then correct":       ["", "482913"],
100         "two wrong then correct":    ["111111", "222222", "482913"],
101         "three wrong":               ["1", "2", "3"],
102         "blanks between wrong ones": ["1", "", "2", "", "3", "482913"],
103     }
104     print(f"{'case':28} {'while':32} {'for':32}")
105     for name, e in cases.items():
106         print(f"{name:28} {verify_while(OTP, e):32} {verify_for(OTP, e):32}")
107         assert verify_while(OTP, e) == verify_for(OTP, e)
108
109     print("\nChanged condition (5 attempts if security question correct):")
110     wrong5 = ["1", "2", "3", "4", "482913"]
111     print("  security OK    :", verify_extended(OTP, wrong5, True))
112     print("  security FAILED:", verify_extended(OTP, wrong5, False))
113
114     print("\nRecursive version :", verify_recursive(OTP, ["", "1", "482913"]))
115     print("Counter-in-wrong-place bug on ['', '', '', '482913']:", verify_bug(OTP, ["", "", "", "482913"]))
116     print("Correct version on the same input            :", verify_while(OTP, ["", "", "", "482913"]))
```

Execution output screenshot

Q2 - Execution output

```
1   $ python q2_otp.py
2   case                         while                           for
3   correct first time           VERIFIED on attempt 1           VERIFIED on attempt 1
4   blank, then correct          VERIFIED on attempt 1           VERIFIED on attempt 1
5   two wrong then correct       VERIFIED on attempt 3           VERIFIED on attempt 3
6   three wrong                  LOCKED for 30 minutes           LOCKED for 30 minutes
7   blanks between wrong ones    LOCKED for 30 minutes           LOCKED for 30 minutes
8
9   Changed condition (5 attempts if security question correct):
10    security OK    : VERIFIED on attempt 5 (limit was 5)
11    security FAILED: LOCKED for 30 minutes
12
13  Recursive version : VERIFIED on attempt 2
14  Counter-in-wrong-place bug on ['', '', '', '482913']: LOCKED (wrongly - blanks were counted)
15  Correct version on the same input            : VERIFIED on attempt 1
```

Q3 – Engineer a Vehicle Registration Number Validator

Problem – Toll plaza number-plate records. The same vehicle is typed in many ways (tn 09 ab 1234, TN-09-AB-1234, Tn09Ab1234). `normalise()` removes all spaces and hyphens and converts to upper case with string methods only. `is_valid()` accepts a length of 9 or 10 and then checks each part by slicing: `reg[:2]` letters (state), `reg[2:4]` digits (district), `reg[4:-4]` one or two letters (series) and `reg[-4:]` four digits. `format_reg()` prints a valid number as TN 09 AB 1234.

Inquiry. Strings are immutable: their characters cannot be changed in place, which lets Python share, hash and reuse them safely. `text.upper()` does not change `text`; it returns a new string, so the result has to be assigned. For a 10-character number `s[-4:]` is the last four digits and `s[4:-4]` is the two-letter series; for a 9-character number `s[-4:]` is again the last four characters and `s[4:-4]` is the one-letter series. Slicing from both ends therefore handles both lengths with the same code.

Alternative concept and complexity. `re.fullmatch(r"[A-Z]{2}[0-9]{2}[A-Z]{1,2}[0-9]{4}", reg)` gives the same answers on all six designed inputs (asserted in the program). Both approaches take O(L) time for a number of length L and O(1) extra space. The regular expression is shorter and easier to change if the pattern changes; the string-method version is easier to step through and explain, and is what the question requires.

Six designed inputs (predictions written before running)

Input	Normalised	Predicted	Observed	Reason
Tn09Ab1234	TN09AB1234	valid	valid	valid: 2 letters, 2 digits, 2 letters, 4 digits
TN-09-AB-1234	TN09AB1234	valid	valid	valid after removing hyphens
KA 01 M 0007	KA01M0007	valid	valid	valid: one-letter series
TN09ABC1234	TN09ABC1234	rejected	rejected	rejected: 3-letter series (length 11)
T109AB1234	T109AB1234	rejected	rejected	rejected: digit in the state code
TN09AB123	TN09AB123	rejected	rejected	rejected: only three final digits

Code screenshot

Q3 - Code

```
"""Q3 - Vehicle registration number validator (string operations only, no regex)"""

def normalise(text):
    """'tn 09 ab 1234' -> 'TN09AB1234' (spaces and hyphens removed, upper case)."""
    # TODO 1 completed
    return "".join(text.split()).replace("-", "").upper()


def is_valid(reg):
    """Valid form: 2 letters, 2 digits, 1 or 2 letters, 4 digits (e.g. TN09AB1234)."""
    # TODO 2 completed
    if len(reg) not in (9, 10):
        return False
    state, district = reg[:2], reg[2:4]
    series, number = reg[4:-4], reg[-4:]      # series is 1 or 2 characters
    return (state.isalpha() and state.isascii()
            and district.isdigit() and district.isascii()
            and series.isalpha() and series.isascii()
            and number.isdigit() and number.isascii())


def format_reg(reg):
    """Format a VALID number for the receipt: TN09AB1234 -> 'TN 09 AB 1234'."""
    if not is_valid(reg):
        raise ValueError("cannot format an invalid number: " + reg)
    return " ".join([reg[:2], reg[2:4], reg[4:-4], reg[-4:]])


```

Q3 - Code

```python
def is_valid_regex(reg):                    # alternative concept (comparison only)
    import re
    return re.fullmatch(r"[A-Z]{2}[0-9]{2}[A-Z]{1,2}[0-9]{4}", reg) is not None


if __name__ == "__main__":
    for raw in ["tn 09 ab 1234", "KA-01-M-0007", "TN9AB1234", "TN 22 1234 AB"]:
        reg = normalise(raw)
        print(f"{raw:15} -> {reg:11} {'valid' if is_valid(reg) else 'INVALID'}")

    print("\nFormatted:", format_reg(normalise("tn 09 ab 1234")))

    # six inputs I designed, with predictions made before running
    tests = [  # (input, predicted valid?)
        ("Tn09Ab1234",     True),    # mixed case, no separators
        ("TN-09-AB-1234",  True),    # hyphens
        ("KA 01 M 0007",   True),    # one-letter series
        ("TN09ABC1234",    False),   # 3-letter series -> length 11 -> REJECT
        ("T109AB1234",     False),   # digit in state code -> REJECT
        ("TN09AB123",      False),   # only 3 final digits -> length 9 but 'B123' fails
    ]
    print(f"\n{'input':16} {'normalised':12} predicted  observed  regex")
    for raw, pred in tests:
        reg = normalise(raw)
        got = is_valid(reg)
        assert got == pred == is_valid_regex(reg), raw
        print(f"{raw:16} {reg:12} {str(pred):9}  {str(got):8}  {is_valid_regex(reg)}")

    print("\nImmutability: s = 'tn09'; s.upper() ->", "tn09".upper(), "| s is still", "tn09")
    for s in ("TN09AB1234", "TN9AB1234"):
        print(f"{s} ({len(s)} chars): s[-4:] = {s[-4:]!r}, s[4:-4] = {s[4:-4]!r}")
```

Execution output screenshot

Q3 - Execution output

```
$ python q3_plate.py
tn 09 ab 1234   -> TN09AB1234  valid
KA-01-M-0007    -> KA01M0007   valid
TN9AB1234       -> TN9AB1234   INVALID
TN 22 1234 AB   -> TN221234AB  INVALID

Formatted: TN 09 AB 1234

input            normalised   predicted  observed  regex
Tn09Ab1234       TN09AB1234   True       True      True
TN-09-AB-1234    TN09AB1234   True       True      True
KA 01 M 0007     KA01M0007    True       True      True
TN09ABC1234      TN09ABC1234  False      False     False
T109AB1234       T109AB1234   False      False     False
TN09AB123        TN09AB123    False      False     False

Immutability: s = 'tn09'; s.upper() -> TN09 | s is still tn09
TN09AB1234 (10 chars): s[-4:] = '1234', s[4:-4] = 'AB'
TN9AB1234 (9 chars): s[-4:] = '1234', s[4:-4] = 'B'
```

Q4 – Building Long Text: + or join()?

Problem – Monthly SMS bill statement. A mobile operator builds each statement by adding one line per call or SMS to a string, and generation is slow for heavy users. Three implementations build the same text: (a) repeated += in a loop, (b) a list of parts followed by `join()`, and (c) an f-string for each line collected with join; `io.StringIO` is the alternative concept. The program asserts that all of them produce identical text (length 3,048,890 at 200,000 lines) and prints the measured times at 1,000, 10,000 and 200,000 lines.

Prediction and explanation. join is predicted to be fastest: it measures all the parts first and copies every character once, O(n). Strings are immutable, so each += creates a new string and copies the old content; in the worst case this copies up to $O(n^2)$ characters. CPython sometimes optimises += when the string has no other reference, which is why the measured gap at 200,000 lines is smaller than the theory suggests; join remains the recommended practice because it does not depend on that optimisation. The f-string only changes how each line is formatted, so (c) still needs join to collect the lines and is not faster than join over ready-made parts.

Alternative concept and complexity. Writing each line to `io.StringIO` (or straight to a file) is O(n) time and avoids one huge intermediate string; memory is O(n) for the buffer, or O(1) when lines are written directly to a file. Times differ from one computer to another: the output below was measured when this report was generated, and the program should be run again on the submitting computer (it reports the average of three runs).

Code screenshot

Q4 - Code

```python
"""Q4 - Building long text: + or join()?  (Monthly SMS bill statement)"""
import io
import time

items = [f"Item{i}:Rs{i % 50}" for i in range(200_000)]


def build_plus(parts):
    text = ""
    for p in parts:
        text += p + ";"
    return text


def build_join(parts):
    # TODO 1 completed: same text using a single str.join() call
    return ";".join(parts) + ";"


def build_fstring(parts):                    # (c) an f-string for each line, then collected
    lines = [f"{p};" for p in parts]
    return "".join(lines)


def build_stringio(parts):                   # alternative concept: io.StringIO buffer
    buf = io.StringIO()
    for p in parts:
        buf.write(p)
        buf.write(";")
    return buf.getvalue()


def timed(fn, parts):
    start = time.perf_counter()
    out = fn(parts)
    return out, time.perf_counter() - start


if __name__ == "__main__":
    # --- the program given in the question ---
```

Q4 - Code

```
    start = time.perf_counter()
    a = build_plus(items)
    t_plus = time.perf_counter() - start

    start = time.perf_counter()
    b = build_join(items)
    t_join = time.perf_counter() - start

    print("same text:", a == b, "| length:", len(b))
    print(f"+= time: {t_plus:.4f} s   join time: {t_join:.4f} s")

    # --- run three times and report the average of each ---
    runs = 3
    tp = tj = 0.0
    for _ in range(runs):
        tp += timed(build_plus, items)[1]
        tj += timed(build_join, items)[1]
    print(f"\nAverage of {runs} runs @200,000 lines:  += {tp/runs:.4f} s   join {tj/runs:.4f} s")

    # --- measured times at three sizes for all implementations ---
    print(f"\n{'lines':>8} {'+=':>10} {'join':>10} {'f-string':>10} {'StringIO':>10}")
    for n in (1_000, 10_000, 200_000):
        parts = items[:n]
        row = []
        for fn in (build_plus, build_join, build_fstring, build_stringio):
            best = min(timed(fn, parts)[1] for _ in range(3))
            row.append(best)
        assert build_plus(parts) == build_join(parts) == build_fstring(parts) == build_stringio(parts)
        print(f"{n:>8} " + " ".join(f"{t:>10.5f}" for t in row))
```

Execution output screenshot

Q4 - Execution output

```
$ python q4_sms_bill.py
same text: True | length: 3048890
+= time: 0.0154 s   join time: 0.0066 s

Average of 3 runs @200,000 lines:  += 0.0150 s   join 0.0050 s

   lines         +=       join   f-string   StringIO
    1000    0.00007    0.00001    0.00005    0.00005
   10000    0.00066    0.00009    0.00049    0.00054
  200000    0.01374    0.00504    0.01707    0.01233
```

Q5 – Functions, Parameters and Scope

Bug – College canteen order system. In `add_item_buggy(item, order=[])` the default list is created once, when the function is defined, not each time it is called, so every call that omits `order` appends to the same list. The first output line shows both calls returning the same list `['Tea', 'Vada']`, and `add_item_buggy.__defaults__` shows that list now holding the items. **Fix:** use `order=None` and create a new empty list inside the function.

Parameters. Positional arguments are matched by position, keyword arguments by name (in any order), and default parameters make an argument optional and should be immutable values. **Returning two values:** a function can return a tuple such as (total, gst), which the caller unpacks. **Scope:** a name assigned inside a function is local; `nonlocal` rebinds a name of an enclosing function; `global` is needed to rebind a module-level name, otherwise Python treats the name as local and raises `UnboundLocalError` (shown in the output). Changing globals inside functions is usually avoided because the function then depends on hidden state, is harder to test and can be affected by any other code.

Test. Three customers are served in sequence and each order stays independent (asserted). **Alternative concept and complexity.** `make_order()` returns a closure that keeps its own order list, so no list is passed by the caller. Adding an item is O(1) time in both designs (the closure returns a copy of the order for safety, which costs O(m) for m items; remove the copy if that matters). Space is O(m) for each customer's order plus one small closure per customer.

Code screenshot

Q5 - Code

```python
"""Q5 - Functions, parameters and scope (College Canteen Order System)"""

# ---------- the bug ----------
def add_item_buggy(item, order=[]):          # ONE list is created when 'def' runs
    order.append(item)
    return order


# ---------- the fix ----------
def add_item(item, order=None):
    # TODO 1 completed: new empty list when order is None, append and return it
    if order is None:
        order = []
    order.append(item)
    return order


# ---------- 1. positional, keyword and default parameters ----------
def price_line(item, qty=1, rate=10):
    return f"{item} x{qty} @Rs{rate} = Rs{qty * rate}"

# ---------- 2. returning more than one value (total and GST) ----------
def bill_with_gst(subtotal, gst_percent=5):
    gst = round(subtotal * gst_percent / 100, 2)
    return subtotal + gst, gst                # a tuple: (total, gst)

# ---------- 3. local, global and nonlocal ----------
x = "global"
def scope_demo():
    x = "local"                               # a NEW local name; global x untouched
    def inner():
        nonlocal x                            # rebinds scope_demo's x
        x = "changed by nonlocal"
    inner()
    return x

# ---------- 4. changing a global needs the 'global' keyword ----------
orders_served = 0
def serve_bad():
    try:
        orders_served += 1                    # UnboundLocalError: treated as local
```

Q5 - Code

```
    except UnboundLocalError as e:
        return "UnboundLocalError: " + str(e)

def serve_good():
    global orders_served
    orders_served += 1

# ---------- alternative concept: closure keeps each customer's order ----------
def make_order():
    order = []
    def add(item):
        order.append(item)
        return list(order)                    # copy, so callers cannot tamper with it
    return add


if __name__ == "__main__":
    print(add_item_buggy("Tea"), add_item_buggy("Vada"))
    print(add_item("Tea"), add_item("Vada"))
    print("add_item_buggy.__defaults__ after the calls:", add_item_buggy.__defaults__)
    print("add_item.__defaults__:", add_item.__defaults__)

    print("\n-- parameter kinds --")
    print(price_line("Tea"))                          # default qty and rate
    print(price_line("Vada", 3))                      # positional
    print(price_line("Samosa", rate=15, qty=2))       # keyword (any order)

    total, gst = bill_with_gst(200)
    print(f"\ntotal={total} gst={gst}   (tuple unpacking)")

    print("\n-- scope --")
    print("scope_demo() ->", scope_demo(), "| global x is still:", x)
    print(serve_bad())
    serve_good(); serve_good()
    print("orders_served after 2 calls to serve_good:", orders_served)

    print("\n-- three customers in sequence (each order independent) --")
    o1 = add_item("Tea");   o1 = add_item("Vada", o1)
    o2 = add_item("Coffee")
    o3 = add_item("Samosa"); o3 = add_item("Juice", o3)
    print(o1, o2, o3)
    assert o1 == ["Tea", "Vada"] and o2 == ["Coffee"] and o3 == ["Samosa", "Juice"]
    assert o1 is not o2 is not o3

    print("\n-- closure design --")
    c1, c2 = make_order(), make_order()
    c1("Tea"); c2("Coffee")
    print(c1("Vada"), c2("Samosa"))
```

Execution output screenshot

Q5 - Execution output

```
$ python q5_canteen.py
['Tea', 'Vada'] ['Tea', 'Vada']
['Tea'] ['Vada']
add_item_buggy.__defaults__ after the calls: (['Tea', 'Vada'],)
add_item.__defaults__: (None,)

-- parameter kinds --
Tea x1 @Rs10 = Rs10
Vada x3 @Rs10 = Rs30
Samosa x2 @Rs15 = Rs30

total=210.0 gst=10.0   (tuple unpacking)
```

Q5 - Execution output

```
13
14  -- scope --
15  scope_demo() -> changed by nonlocal | global x is still: global
16  UnboundLocalError: cannot access local variable 'orders_served' where it is not associated with a value
17  orders_served after 2 calls to serve_good: 2
18
19  -- three customers in sequence (each order independent) --
20  ['Tea', 'Vada'] ['Coffee'] ['Samosa', 'Juice']
21
22  -- closure design --
23  ['Tea', 'Vada'] ['Coffee', 'Samosa']
```

Q6 – Recursion and Its Limits

Problem – counting ways to climb temple steps. The base case is `n <= 1` returning 1; the recursive case is `ways(n) = ways(n-1) + ways(n-2)`. Hand-checked values for n = 1 to 6 are 1, 2, 3, 5, 8, 13 and every version matches them (asserted). The call tree printed for n = 5 shows `ways(3)` computed twice and `ways(2)` three times: plain recursion repeats work. Memoisation with `functools.lru_cache` remembers each result the first time it is computed, so each value from 0 to n is computed once: 26 calls for n = 25 instead of 242,785.

Growth. Plain calls grow by about 1.618 times for every extra step, $O(\varphi^n)$; memoised calls grow as n + 1, O(n).

Deployment decision (n = 1,000). `sys.getrecursionlimit()` is 1000 and `ways_memo(1000)` needs about 1,000 nested calls, so Python raises `RecursionError` to protect the interpreter from a stack overflow. Raising the limit with `sys.setrecursionlimit(5000)` works here (the answer equals the loop answer) but a limit set too high can crash the interpreter, so it is only a partial fix. Rewriting the function with a loop avoids both problems. **Decision:** use the loop for 1,000 steps (or use both: raise the limit modestly and keep the cache). Plain recursion for n = 1,000 would need about 10^{208} calls and can never finish.

Alternative concept and complexity. A loop that keeps only the last two values takes O(n) time and O(1) space, compared with O(n) cache plus O(n) stack for memoised recursion and $O(\varphi^n)$ time for plain recursion.

Growth investigation (function calls)

n	Plain recursion calls	Memoised calls	Reduction
10	177	11	16 x fewer
20	21,891	21	1,042 x fewer
25	242,785	26	9,338 x fewer
30	2,692,537	31	86,856 x fewer

Code screenshot

Q6 - Code

```python
"""Q6 - Recursion and its limits (Counting ways to climb temple steps)"""
import sys
from functools import lru_cache

calls = 0


def ways_naive(n):
    """Ways to climb n temple steps taking 1 or 2 steps at a time."""
    global calls
    calls += 1

    if n <= 1:
        return 1

    return ways_naive(n - 1) + ways_naive(n - 2)


@lru_cache(maxsize=None)
def ways_memo(n):
    global calls
    calls += 1

    # TODO 1 completed: same base case and recursive case as ways_naive, calling ways_memo
    if n <= 1:
        return 1
    return ways_memo(n - 1) + ways_memo(n - 2)


def ways_loop(n):
    """Alternative concept: keep only the last two values - O(n) time, O(1) space."""
```

Q6 - Code

```python
    prev, cur = 1, 1                # ways(0), ways(1)
    for _ in range(n - 1):
        prev, cur = cur, prev + cur
    return cur


def call_tree(n, depth=0):
    """Print the call tree of the plain recursion (used for n = 5)."""
    print("    " * depth + f"ways({n})")
    if n > 1:
        call_tree(n - 1, depth + 1)
        call_tree(n - 2, depth + 1)


if __name__ == "__main__":
    # 1. correctness by hand, n = 1..6  (1, 2, 3, 5, 8, 13)
    by_hand = {1: 1, 2: 2, 3: 3, 4: 5, 5: 8, 6: 13}
    for n, want in by_hand.items():
        assert ways_naive(n) == ways_loop(n) == want
    print("n = 1..6 by hand:", list(by_hand.values()), "-> all versions agree")

    # 2. call tree for n = 5
    print("\nCall tree for n = 5  (note ways(3) is computed twice, ways(2) three times):")
    call_tree(5)

    # 3. expected output of the question
    calls = 0
    print("\nnaive:", ways_naive(25), "calls =", calls)
    calls = 0
    ways_memo.cache_clear()
    print("memo :", ways_memo(25), "calls =", calls)

    # 4. growth investigation
    print(f"\n{'n':>3} {'naive calls':>12} {'memo calls':>11}")
    for n in (10, 20, 25, 30):
        calls = 0; ways_naive(n); cn = calls
        ways_memo.cache_clear(); calls = 0; ways_memo(n); cm = calls
        print(f"{n:>3} {cn:>12} {cm:>11}")

    # 5. deployment decision: n = 1000
    print("\nsys.getrecursionlimit() =", sys.getrecursionlimit())
    ways_memo.cache_clear()
    try:
        ways_memo(1000)
        print("ways_memo(1000) worked without error")
    except RecursionError as e:
        print("ways_memo(1000) -> RecursionError:", e)

    loop_answer = ways_loop(1000)
    print("ways_loop(1000) has", len(str(loop_answer)), "digits, no error")

    ways_memo.cache_clear()
    old = sys.getrecursionlimit()
    sys.setrecursionlimit(5000)              # option 1: raise the limit
    try:
        memo_answer = ways_memo(1000)
        print("memo with limit 5000 equals loop answer:", memo_answer == loop_answer)
    finally:
        sys.setrecursionlimit(old)
```

Execution output screenshot

Q6 - Execution output

```
$ python q6_steps.py
n = 1..6 by hand: [1, 2, 3, 5, 8, 13] -> all versions agree

Call tree for n = 5  (note ways(3) is computed twice, ways(2) three times):
ways(5)
    ways(4)
        ways(3)
            ways(2)
                ways(1)
                ways(0)
            ways(1)
        ways(2)
            ways(1)
            ways(0)
    ways(3)
        ways(2)
            ways(1)
            ways(0)
        ways(1)

naive: 121393 calls = 242785
memo : 121393 calls = 26

  n  naive calls  memo calls
 10          177          11
 20        21891          21
 25       242785          26
 30      2692537          31

sys.getrecursionlimit() = 1000
ways_memo(1000) -> RecursionError: maximum recursion depth exceeded
ways_loop(1000) has 209 digits, no error
memo with limit 5000 equals loop answer: True
```

Q7 – Diagnose, Redesign and Validate

Program selected from my Q1–Q6 evidence: Q6, `ways_naive(n)`. The redesign is not a built-in function; it follows from the diagnosis below.

A. Diagnose the bottleneck Most of the work is done in the leaf calls: for n = 25 the base cases are reached 121,393 times, and every inner call spends one addition joining two sub-results. The count that grows with the input is the number of calls: 177 (n = 10), 21,891 (n = 20), 242,785 (n = 25) and 2,692,537 (n = 30), about 1.618 times more for each extra step. Work is unnecessarily repeated: `ways(1)` alone is computed 75,025 times for n = 25.

B. Define the design objective For n = 25 reduce the work from 242,785 calls to at most n operations; make n = 30 run in well under 1 ms; and return ways(1000) with no RecursionError and no raised recursion limit.

C. Design the revised solution *What changed:* recursion became a loop that carries ways(i−1) and ways(i−2) forward. *Property exploited:* overlapping sub-problems, and each value depends only on the previous two. *Extra memory:* none beyond two integers, O(1), versus an O(n) cache and O(n) stack for memoisation. *New trade-off:* no cache is shared between separate queries (each query is O(n) again) and very large integers make each addition cost O(digits).

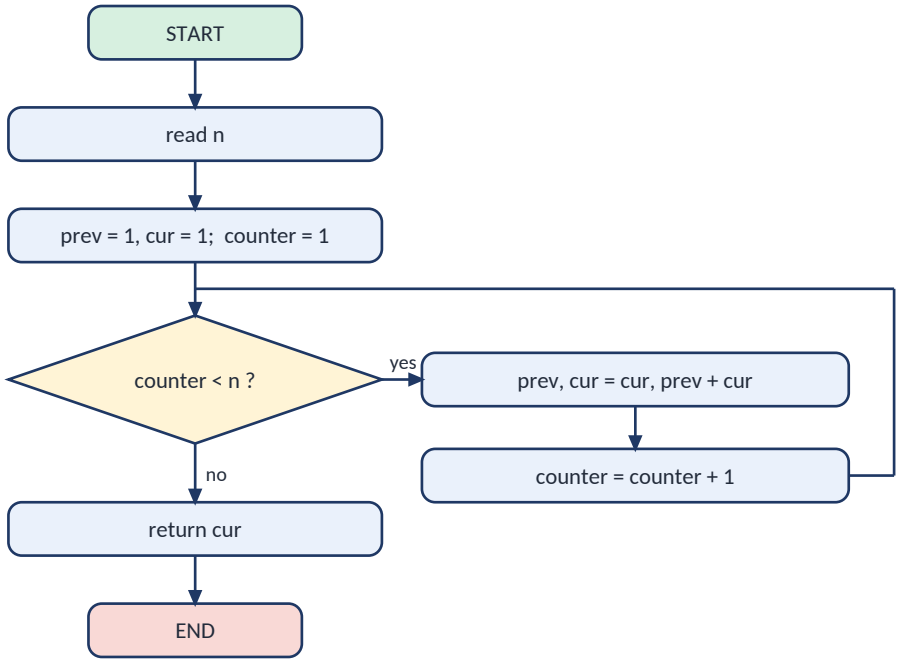
D. Predict before implementing For n = 25 the work falls from 242,785 calls to 24 loop steps (about 10,000 times fewer); the time for n = 30 falls from hundreds of milliseconds to microseconds; n = 1,000 works.

E. Implement and validate The table compares original and redesigned versions with the same inputs and counts; the output panel also shows the timings measured on the computer used to generate this report (run it again on the submitting computer).

Validation table (same inputs, same counts)

n	Original calls	Redesigned steps	Fewer operations	Same result
10	177	9	20 x	Yes
20	21,891	19	1,152 x	Yes
25	242,785	24	10,116 x	Yes
30	2,692,537	29	92,846 x	Yes

Pseudocode and flowchart



F. Defend the final design *Objective achieved:* yes; for n = 25 the calls fell from 242,785 to 24 and n = 1,000 now works. *Size of the improvement:* it grows with n, from $O(\varphi^n)$ time to O(n) time and O(1) memory. *New trade-off:* no

reuse across queries (memoisation answers a repeated query instantly) and large integers cost more per addition. *Decision:* for a trust that prints every n up to 1,000 once, keep the loop, or generate all values in one pass and store them in a list. A redesign that did not outperform the original would still be a valid result if the measurements and the explanation are honest, as the question says.

Code screenshot

Q7 - Code

```
"""Q7 - Diagnose, redesign and validate

Program selected from my Q1-Q6 evidence: Q6 ways_naive(n)  (temple steps).
ORIGINAL : plain recursion.      REDESIGN : bottom-up loop that keeps only the last two values.
(The redesign is NOT a built-in; it comes from the diagnosis below.)

A. DIAGNOSIS (from my own Q6 numbers)
   * Most work is done in the leaves: ways(1)/ways(0) base cases are reached ~121,393 times
     for n = 25, and every inner call spends one addition to join two sub-results.
   * The count that grows with the input is the NUMBER OF CALLS: 177 (n=10), 21,891 (n=20),
     242,785 (n=25), 2,692,537 (n=30). Each +1 in n multiplies the calls by about 1.618.
   * Work IS repeated: ways(3) is computed 2 times and ways(2) 3 times in the n = 5 tree;
     for n = 25 ways(1) alone is computed 75,025 times.

B. OBJECTIVE (measurable)
   For n = 25 reduce calls from 242,785 to <= n (25), make n = 30 run in well under 1 ms, and
   return ways(1000) with no RecursionError and no raised recursion limit.

C. REDESIGN
   Changed   : recursion -> loop; the two values ways(i-1), ways(i-2) are carried forward.
   Property  : overlapping sub-problems + each ways(i) needs ONLY the previous two values.
   Extra mem : O(1) (two integers) versus O(n) cache + O(n) stack for memoisation.
   Trade-off : no cache shared between separate queries (each query is O(n) again), and the
               numbers become huge big-ints for large n (the additions cost O(digits)).

   Pseudocode
       function ways_fast(n):
           prev <- 1 ; cur <- 1
           repeat (n - 1) times:
               (prev, cur) <- (cur, prev + cur)
           return cur

   Flowchart (text form)
       START -> read n -> prev = 1, cur = 1 -> counter = 1
       -> counter < n ?
            yes -> prev, cur = cur, prev + cur -> counter = counter + 1 -> (back to the test)
            no  -> return cur -> END

D. PREDICTION (made before implementing)
   Calls for n = 25 fall from 242,785 to 24 loop steps (about 10,000x fewer); time for n = 30
   falls from tens/hundreds of milliseconds to microseconds; n = 1000 works.
"""
import sys
import time

calls = 0


def ways_original(n):
    global calls
    calls += 1
    if n <= 1:
        return 1
    return ways_original(n - 1) + ways_original(n - 2)


steps = 0

```

Q7 - Code

```
59
60  def ways_redesigned(n):
61      global steps
62      prev, cur = 1, 1
63      for _ in range(n - 1):
64          steps += 1                    # one loop pass = one 'call' of work
65          prev, cur = cur, prev + cur
66      return cur
67
68
69  def timed(fn, n):
70      start = time.perf_counter()
71      result = fn(n)
72      return result, time.perf_counter() - start
73
74
75  if __name__ == "__main__":
76      print("E. VALIDATION - same inputs, same counts, same timing method\n")
77      print(f"{'n':>3} {'orig calls':>11} {'new steps':>10} {'orig time (s)':>14}",
78            f"{'new time (s)':>13} {'speed-up':>10}  same result")
79      for n in (10, 20, 25, 30):
80          calls = 0; steps = 0
81          r1, t1 = timed(ways_original, n)
82          r2, t2 = timed(ways_redesigned, n)
83          print(f"{n:>3} {calls:>11} {steps:>10} {t1:>14.6f} {t2:>13.8f} {t1 / t2:>9.0f}x  {r1 == r2}")
84
85      print("\nRequirement change: n = 1000")
86      try:
87          ways_original(1000)
88      except RecursionError:
89          print("  original  -> RecursionError (limit", sys.getrecursionlimit(), ")")
90      print("  original would also need about 1.6**1000 ≈ 10^208 calls: it would never finish")
91      r, t = timed(ways_redesigned, 1000)
92      print(f"  redesigned -> {len(str(r))}-digit answer in {t:.6f} s, no recursion limit touched")
93
94      print("""
95  F. DEFEND THE FINAL DESIGN
96     * Objective achieved?   Calls for n=25 dropped from 242,785 to 24 (>10,000x); n=1000 now works.
97     * Size of improvement   Grows with n: time went from O(phi^n) to O(n), memory is O(1).
98     * New trade-off         No reuse across queries (memoisation answers a repeat query
99                             instantly; the loop recomputes), and huge integers cost more per add.
100    * Would I change it?    For a trust printing every n up to 1000 once, keep the loop, or
101                            generate all values in ONE pass and store them in a list.
102    * If it did NOT beat    A redesign that does not outperform the original would still be a
103      the original?         valid result if the measurements and the explanation are honest
104                            (the question says so explicitly).""")
```

Execution output screenshot

Q7 - Execution output

```
1   $ python q7_redesign.py
2   E. VALIDATION - same inputs, same counts, same timing method
3
4     n  orig calls  new steps  orig time (s)  new time (s)   speed-up  same result
5    10         177          9       0.000014    0.00000234         6x  True
6    20       21891         19       0.001675    0.00000255       658x  True
7    25      242785         24       0.018637    0.00000479      3890x  True
8    30     2692537         29       0.206047    0.00000553     37240x  True
9
10  Requirement change: n = 1000
11    original  -> RecursionError (limit 1000 )
12    original would also need about 1.6**1000 ≈ 10^208 calls: it would never finish
13    redesigned -> 209-digit answer in 0.000106 s, no recursion limit touched
14
```

Q7 - Execution output

```
F. DEFEND THE FINAL DESIGN
   * Objective achieved?   Calls for n=25 dropped from 242,785 to 24 (>10,000x); n=1000 now works.
   * Size of improvement   Grows with n: time went from O(phi^n) to O(n), memory is O(1).
   * New trade-off         No reuse across queries (memoisation answers a repeat query
                           instantly; the loop recomputes), and huge integers cost more per add.
   * Would I change it?    For a trust printing every n up to 1000 once, keep the loop, or
                           generate all values in ONE pass and store them in a list.
   * If it did NOT beat    A redesign that does not outperform the original would still be a
     the original?         valid result if the measurements and the explanation are honest
                           (the question says so explicitly).
```

Appendix A – README.md

The README that accompanies the GitHub repository (student details, Python version, libraries, how to run and the result of each task).

File

README.md

```
# CSA08 - Python Programming - Assignment 2

- Student name: <YOUR NAME>
- Register number: <YOUR REGISTER NUMBER>
- Python version: 3.12 (works on 3.10+ because Q1 uses match-case)
- Libraries: standard library only - `time`, `io`, `sys`, `functools` (see requirements.txt)

## How to run
    python Q1/q1_subsidy.py      (likewise Q2 ... Q7)

## Task results
| Task | File | Result |
|------|------|--------|
| Q1 | Q1/q1_subsidy.py | if-elif-else matches all 8 predictions; separate ifs gave 2 decisions for (80 units, rati
  on card) |
| Q2 | Q2/q2_otp.py | blank entries never count; 3 wrong attempts -> locked; 5 attempts after correct security answ
  er |
| Q3 | Q3/q3_plate.py | TN 09 AB 1234 and KA 01 M 0007 valid; TN9AB1234 and TN221234AB invalid |
| Q4 | Q4/q4_sms_bill.py | same text, length 3048890; join about 3x faster than += at 200,000 lines |
| Q5 | Q5/q5_canteen.py | buggy: ['Tea','Vada'] ['Tea','Vada']; fixed: ['Tea'] ['Vada'] |
| Q6 | Q6/q6_steps.py | naive 242785 calls vs memo 26; ways_memo(1000) -> RecursionError |
| Q7 | Q7/q7_redesign.py | recursion -> two-variable loop: 242785 calls -> 24 steps for n = 25 |

Times differ from computer to computer: re-run on your own machine and take your own
output screenshots before submitting (output.txt files are included as a reference only).
```

Appendix B – requirements.txt, Git commit plan and repository link

The standard library only is used, so there is nothing to install. The commit plan gives one commit per task, because the GitHub commit history is verified and a single upload on the final day is not accepted.

requirements.txt

requirements.txt

```
# Only the Python standard library is used (time, io, sys, functools).
# No third-party packages are required.
```

Git commands (one commit per task)

git_commands.txt

```
# Commit progressively - one commit per task (the commit history is verified)
git init CSA08-<RegisterNumber>-Assignment2 && cd CSA08-<RegisterNumber>-Assignment2
# after copying Q1 in:
git add Q1 && git commit -m "Task Q1: subsidy rules with if-elif-else"
git add Q2 && git commit -m "Task Q2: OTP attempt loops"
git add Q3 && git commit -m "Task Q3: registration number validator"
git add Q4 && git commit -m "Task Q4: + versus join timing"
git add Q5 && git commit -m "Task Q5: mutable default argument fix and scope"
git add Q6 && git commit -m "Task Q6: recursion, memoisation, recursion limit"
git add Q7 && git commit -m "Task Q7: diagnose, redesign and validate"
git add README.md requirements.txt && git commit -m "Add README and requirements"
```

GitHub repository link

Repository name: CSA08-<RegisterNumber>-Assignment2

Link: _____

GitHub Repository:

https://github.com/harishraghavsk-dot/python-assesment